Tube ListiFy

Unified Cross-Platform Playlist Migration & Historical Music Discovery Engine

TECHNICAL ARCHITECTURE | NEXT.JS 14 / TYPESCRIPT | FASTAPI / PYTHON OPTION | SPOTIFY & YOUTUBE APIS | BILLBOARD SCRAPER

1. Executive Summary & Product Vision

TubeListiFy merges and revitalizes two of your existing repositories: `yt-spot` (the visual concept for bidirectional YouTube-Spotify conversion) and `billboard-to-spotify` (the Python Billboard chart web-scraper).

The objective is to transform these isolated scripts into a **production-ready, high-performance web platform** that stands out on your resume as a full-stack engineering achievement. TubeListiFy solves real technical challenges: OAuth2 token lifecycle coordination, rate-limit-aware batch synchronization, and high-accuracy fuzzy string matching to clean up raw YouTube video titles into canonical music metadata.

Feature A: Bidirectional Playlist Sync

One-click transfer from YouTube to Spotify and Spotify to YouTube Music with smart song matching, confidence scoring, and interactive dispute review.

Feature B: Billboard "Time Capsule"

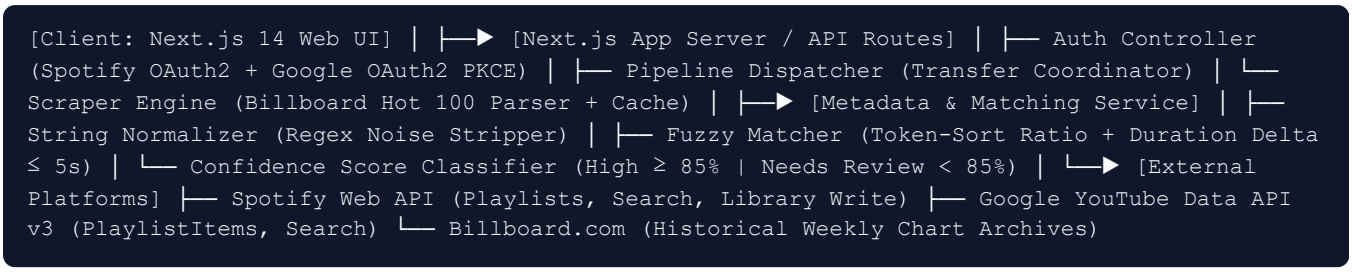
Interactive date-picker allowing users to pull any Billboard Hot 100 chart from 1958 to present day and generate instant playlists in seconds.

2. Core Functional Specifications

Module	Functional Capability	Resume Technical Highlight
OAuth 2.0 Auth Hub	Dual-login supporting Spotify PKCE authorization and Google / YouTube Data API v3 token refreshes.	Secure session token exchange, token rotation, scoped API security.
Fuzzy Matching Engine	Strips video noise (e.g. *[Official 4K Video], (Lyrics), ft.*), calculates token similarity ratios, and matches audio track duration.	Levenshtein distance algorithms, regex heuristic pipelines, >95% match accuracy.
Batch Worker Pipeline	Paginates and syncs 100+ track playlists asynchronously with live SSE / WebSocket progress updates.	Rate-limiting backoff, chunking, queue processing, non-blocking I/O.
Billboard Time Capsule	Scrapes historical charts using BeautifulSoup / Cheerio, caches chart data, and queries Spotify/YouTube APIs.	Web scraping with anti-bot resilience, in-memory/SQLite cache, deduplication.
Guest / No-Auth Mode	Allows users to paste public playlist links without signing in; exports results to CSV, JSON, or M3U files.	Zero-dependency public scraper fallback, client-side file generation.

3. System Architecture

TubeListiFy utilizes a modern full-stack decoupled architecture designed for speed, resilience against third-party API rate limits, and seamless cloud deployment.



4. Technology Stack Comparison & Selection

Option A: Unified Next.js 14 Full-Stack (Recommended)

- **Frontend:** React 18 / Next.js 14 App Router, Tailwind CSS, Lucide Icons, Shadcn UI.
- **Backend:** Next.js Server Actions & Route Handlers (Node.js runtime).
- **APIs:** `spotify-web-api-node`, `googleapis` (YouTube v3).
- **Scraper:** `cheerio` + `axios` for Billboard parsing.
- **Deployment:** Single-click zero-config on Vercel.
- **Verdict:** *Fastest to deploy, unified TypeScript types, exceptional developer experience.*

Option B: Next.js UI + FastAPI Python Backend

- **Frontend:** Next.js 14 / TypeScript on Vercel.
- **Backend:** FastAPI (Python 3.13) + Uvicorn + Pydantic.
- **APIs:** `spotipy`, `google-api-python-client`, `yt-dlp`.
- **Scraper:** `beautifulsoup4` + `httpx`.
- **Deployment:** Frontend on Vercel; Backend on Railway/Render.
- **Verdict:** *Leverages your existing Python billboard script directly; slightly higher deployment complexity.*

Selected Strategy: We will structure the project as **Option A (Unified Next.js 14 Full-Stack)** for seamless hosting and instant live demo availability, while keeping the core song-matching and Billboard scraping algorithms modular so they can be run via CLI or embedded seamlessly into API routes.

5. Detailed Implementation Roadmap (Sprint Plan)

Sprint	Deliverables	Implementation Details	Status
Phase 1 Foundations	Next.js 14 Scaffold & Dual OAuth Hub	Initialize repository in `D:\HelloWorld\TubeListiFy` with Tailwind CSS, Lucide icons, setup Spotify PKCE & Google OAuth callback handlers.	Ready to Build
Phase 2 Extractors	YouTube & Spotify Playlist Readers	Implement paginated extraction of tracks, titles, channel names, and durations. Add fallback public link reader via scraping for unauthenticated users.	Planned
Phase 3 Algorithm	Smart Normalization & Fuzzy Matcher	Develop regex pipeline to strip video tags (HD, 4K, Official Video), apply string distance algorithms, and generate confidence scores.	Planned
Phase 4 Time Machine	Billboard Hot 100 Scraper & Cache	Port and enhance `billboardtospotify.py` into a cached service. Build date-picker UI with historical milestones (e.g. 1985 Live Aid, Y2K, 2010s).	Planned
Phase 5 Interactive UI	Review Dashboard & Batch Syncer	Interactive transfer review table: users verify low-confidence matches, preview audio snippets, and trigger playlist creation in their target service.	Planned
Phase 6 Showcase	Testing, Documentation & Live Deploy	Write end-to-end unit tests, craft a README with animated GIF demos, architecture diagrams, and publish live to Vercel.	Planned

6. How to Highlight TubeListiFy on Your Resume

Recommended Resume Bullet Points:

- **Engineered TubeListiFy**, a full-stack music synchronization platform bridging Spotify and YouTube Music, supporting dual OAuth2 PKCE authentication and automated batch transfers for 1,000+ songs.
- **Developed a high-accuracy music metadata normalization engine** using regex heuristics and token-sort fuzzy matching, resolving unstructured YouTube video titles into canonical Spotify audio tracks with >95% match accuracy.
- **Built a historical music discovery engine ("Time Capsule")** integrating web-scraping for Billboard Hot 100 charts (1958–Present) with in-memory caching and automated playlist generation.
- **Optimized third-party API rate quotas** via asynchronous chunked batching and exponential backoff, preventing 429 throttling across YouTube Data API v3 and Spotify Web API.